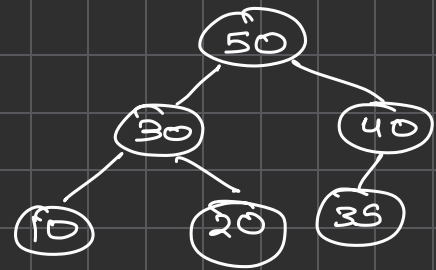


HEAPS, GRAPHS & TREES

⇒ Heap:

- Top element: $O(1)$ (min/max)
- Insert/Delete: $O(\log(n))$
- Build heap: $O(n)$
- Search: $O(n)$



- Conceptually represented as tree's.
- Stored as array

0 1 2 3 4 5
[50, 30, 40, 10, 20, 15]

- If an element is at index 'i'
 - Parent → $(i-1)/2$
 - Left child → $2i+1$
 - Right child → $2i+2$

∴ It's a complete binary tree (i.e., all levels are full, except the last level)

GRAPHS

⇒ BFS, DFS, Dijkstra's, ...

⇒ For BFS & DFS:

$$T.C = O(V+E)$$

$$S.C = O(V)$$

⇒ For Dijkstra's (doesn't work with graph's with -ve weight)

$$T.C = O((V+E) \log V)$$

$$S.C = O(V)$$

PROBLEM - 1

973. K Closest Points to Origin

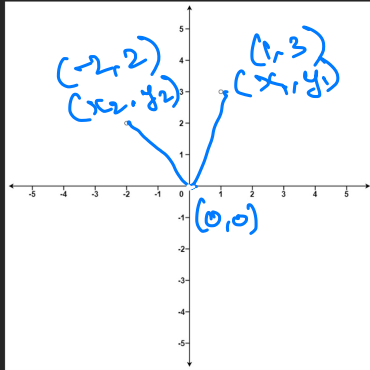
Medium Topics Companies

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in **any order**. The answer is **guaranteed** to be **unique** (except for the order that it is in).

Example 1:



Input: points = [[1,3], [-2,2]], k = 1
Output: [[-2,2]]

$K=1$

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Brute force / Naive:

- 1) find distances of points from origin
- 2) Sort points based on the distances
- 3) return the top K closest

$$T.C = O(n) + O(n \log(n)) + O(K)$$

$$S.C = O(n)$$

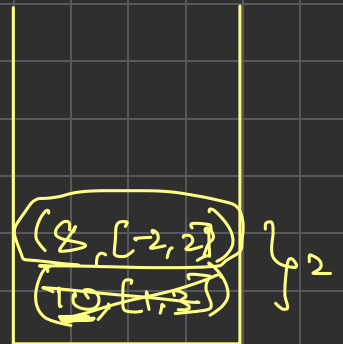
Optimal: max-heap

points = $\begin{bmatrix} 1, 3 \\ -2, 2 \end{bmatrix}$
 $K=1$

$$\begin{aligned} \text{dist} &= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \\ &= \sqrt{x^2 + y^2} \\ \text{dist} &= (x^2 + y^2) \end{aligned}$$

$$\begin{aligned} \text{dist}(0) &= 1^2 + 3^2 \\ &= 10 \end{aligned}$$

$$\begin{aligned} \text{dist}(1) &= (-2)^2 + 2^2 \\ &= 4 + 4 \\ &= 8 \end{aligned}$$



size(max-heap) = 2

$$\begin{aligned} T.C &= O(n \times \log(n)) \\ S.C &= O(n) \end{aligned}$$

PROBLEM - 2

200. Number of Islands

Solved

Medium Topics Companies

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

Input: grid =
 [
 ["1","1","1","1","0"],
 ["1","1","0","1","0"],
 ["1","1","0","0","0"],
 ["0","0","0","0","0"]
]
 Output: 1

Example 2:

Input: grid =
 [
 ["1","1","0","0","0"],
 ["1","1","0","0","0"],
 ["0","0","1","0","0"],
 ["0","0","0","1","1"]
]
 Output: 3

grid =

	0	1	2	3	4
0	1	1	0	0 x	0 x
1	1	1	0 x	0 x	0 x
2	0 x	0 x	1	0 x	0 x
3	0 x	0 x	0 x	1	1

ans = 3

	0	1	2	3	4
0	1	1	0	0	0
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1

(0,0), (2,2), (3,3)

visited =

count = ~~3~~ 3

T.C = $O(m \times n)$
 S.C = $O(m \times n)$

208. Implement Trie (Prefix Tree)

Solved

Medium Topics Companies

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `boolean search(String word)` Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise.
- `boolean startsWith(String prefix)` Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

Example 1:

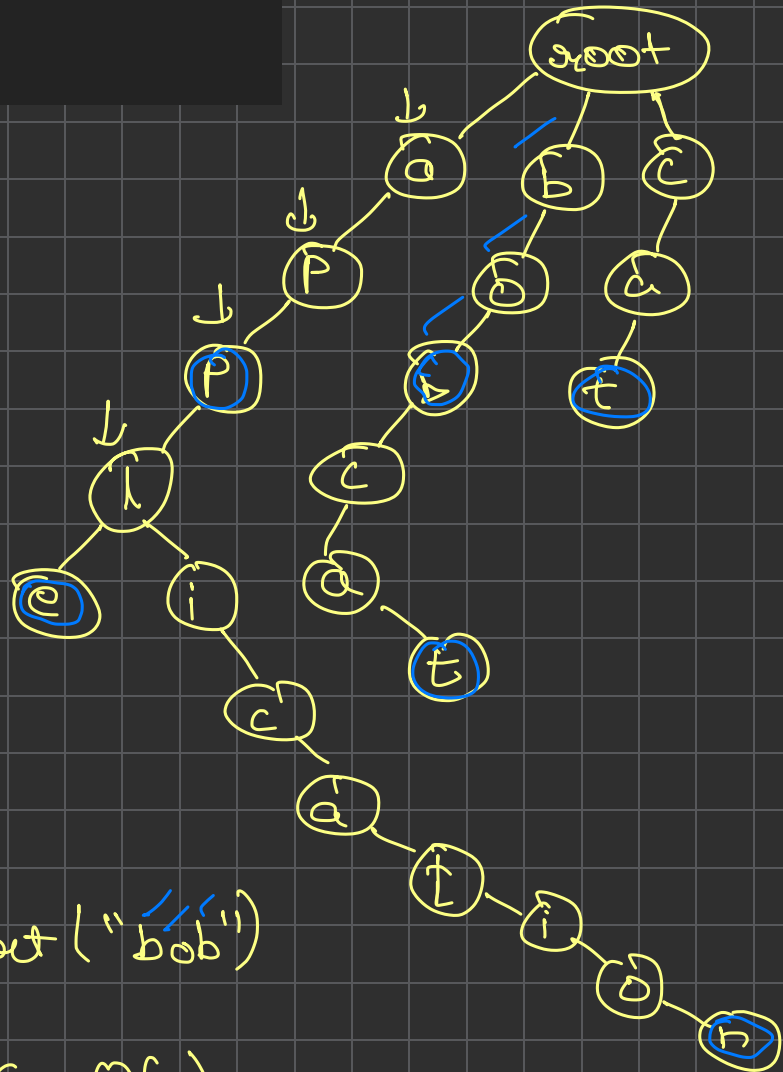
Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]

Output
[null, null, true, false, true, null, true]

Explanation

```
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // return True
trie.search("app"); // return False
trie.startsWith("app"); // return True
trie.insert("app");
trie.search("app"); // return True
```

TrieNode:
children: dict
isEnd: bool



insert("apple")

search("apple")

↑↑↑↑↑

search("app")

↑↑↑↑

startsWith("app")

↑↑↑↑

insert("app")

↑↑↑↑

insert("application")

insert("bobcat")

insert("cat"), insert("bob")

286. Walls and Gates Premium

Solved

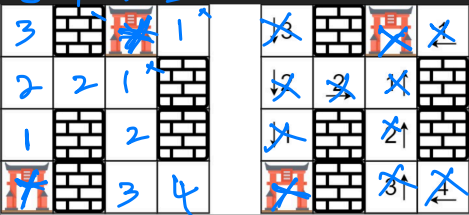
Medium Topics Companies

You are given an $m \times n$ grid `rooms` initialized with these three possible values.

- 1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent `INF` as you may assume that the distance to a gate is less than 2147483647 .

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with `INF`.

Example 1:



$T.C = O(m \times n)$
 $S.C = O(m \times n)$

visited BFS: queue

~~(0,2)~~, ~~(3,0)~~, ~~(0,3)~~, ~~(1,2)~~, ~~(3,0)~~

~~(3,0)~~, ~~(3,2)~~, ~~(1,0)~~, ~~(3,2)~~, ~~(0,2)~~

~~(3,2)~~

Input: rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]
Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]